Unit 8
Shared Memory Transactions

EL-GY 9463: INTRODUCTION TO HARDWARE DESIGN

PROFS. SUNDEEP RANGAN, SIDDHARTH GARG

Learning Objectives

- ❑ Describe the basic components of random access memory
  - Address and data bus, address decoder, storage
- ❑ Identify which types of memory should be used for storage
  - External vs. internal
  - Registers vs. block memory
- ❑ Describe and analyze timing for the AXI4-Lite protocol for reading and writing registers
- ❑ Implement a simple IP with AXI4-Lite control registers using Vitis HLS
- ❑ Synthesize and simulate the IP and analyze the timing diagrams
- ❑ Deploy the IP on an FPGA board and interface with the IP via PYNQ

Outline

- Shared Memory and Interconnects
- Shared Memory Transactions
- AXI4-Memory Mapped Protocol
- Implementing Shared Memory in Vitis HLS

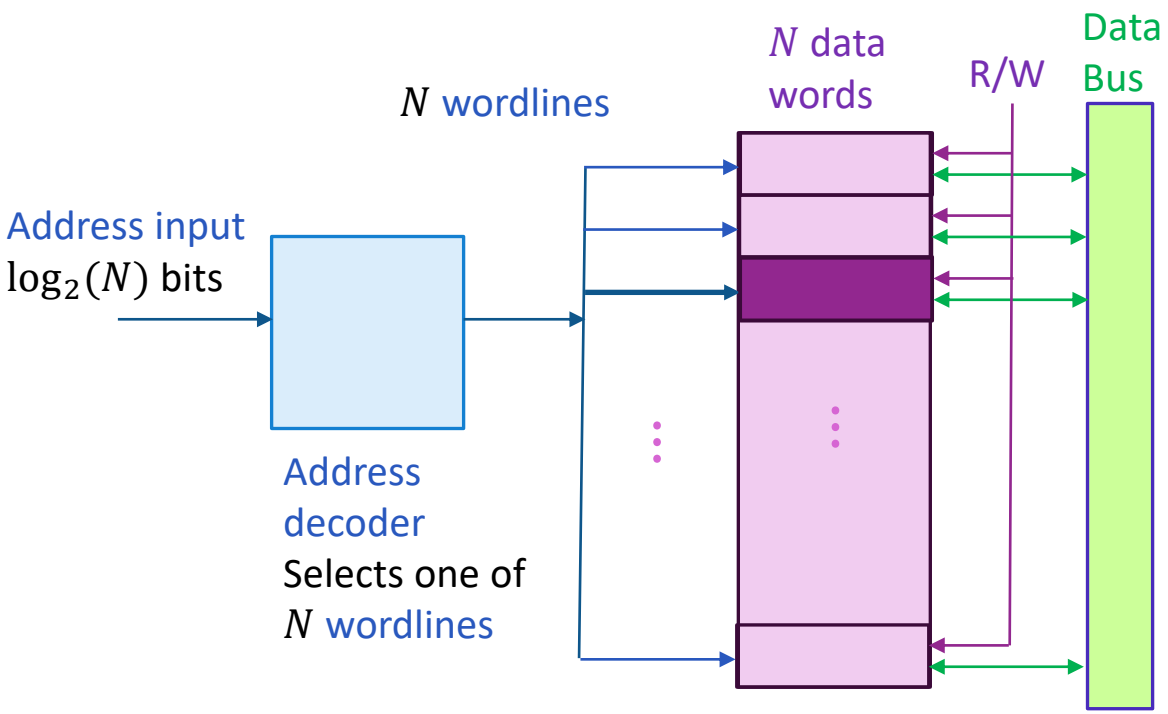
Addressable Memory

- An array of elements in storage
  - Use for vectors, matrices, packets, data, …
- Word: Each of the data element
  - Typically, has word / data width = 32 or 64 bits
- Memory depth = number of words
- Memory size=depth × word width
- Address: A location of a word
  - Often byte-addressable
  - Word k is at byte address $4k$ when word width=32

Byte address	Data
0	Word 0
4	Word 1
8	Word 2
$4(N-1)$	Word $N-1$

Example 32-bit memory with N depth

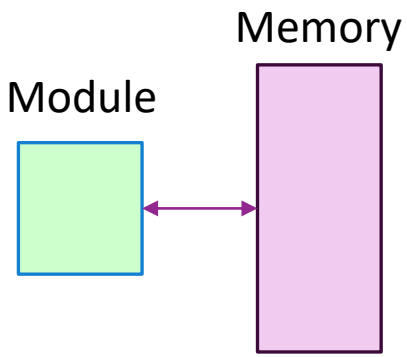
Typical RAM Architecture



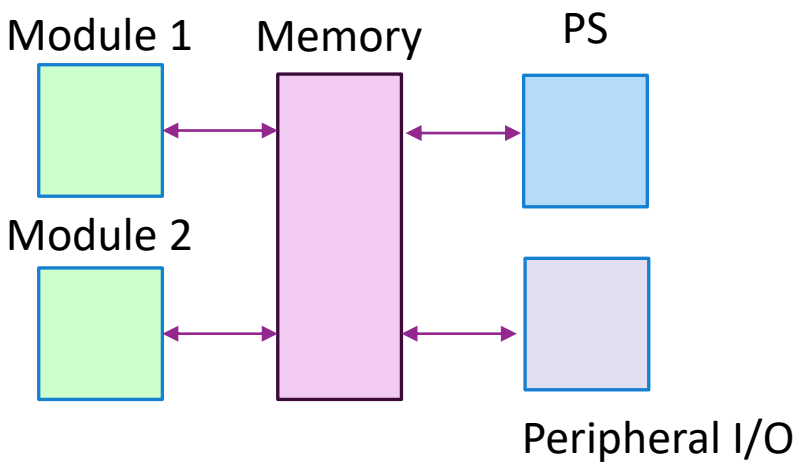
- Three interface signals:
  - Address input: Which word to select
  - Read/Write: Which direction of transfer
  - Data: Value to read or write
- Address input:
  - $\log_2(N)$ bits to select one of N words
  - Each word has a wordline select
  - Selection determined by address decoder

Dedicated vs Shared Memory

- Dedicated memory
  - Memory that is dedicated to a single hardware module

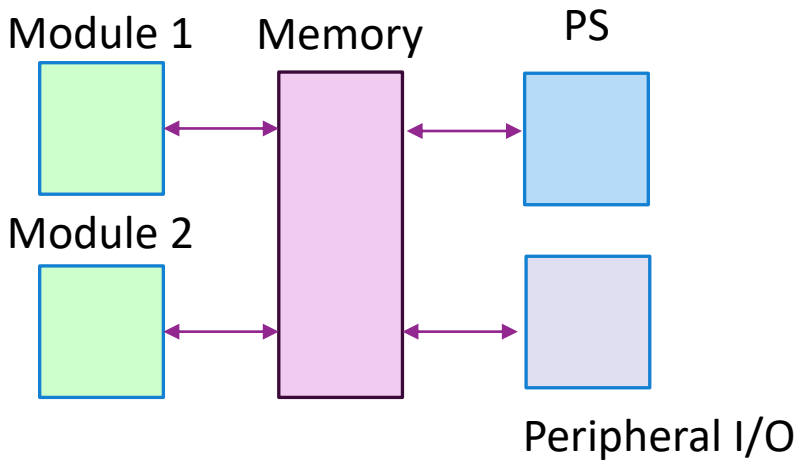


- Shared memory
  - Memory shared between modules
  - Also processing system and peripheral I/O



Why Share Memory?

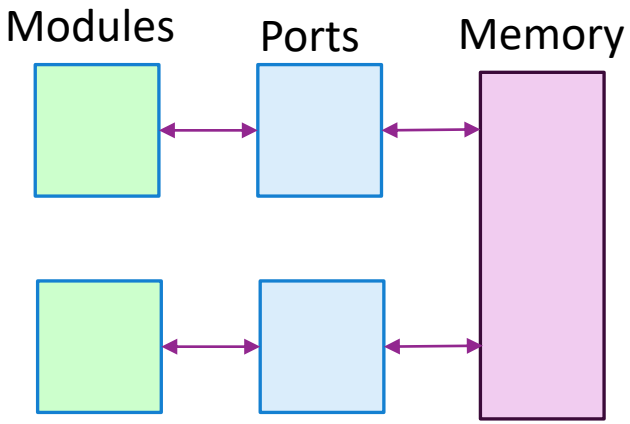
- Multiple modules, I/O, PS need operate on same data
  - Example video pipeline: Network I/O → Filtering → Coding
  - Virtually all complex tasks built over multiple modules
- Enables common memory for large data
  - E.g., ML parameters, data, large video
  - Dedicated memory becomes inefficient
- Allows dynamic memory allocation
  - Use memory as needed
  - Allocate memory to modules as needed



How to Build Shared Memory

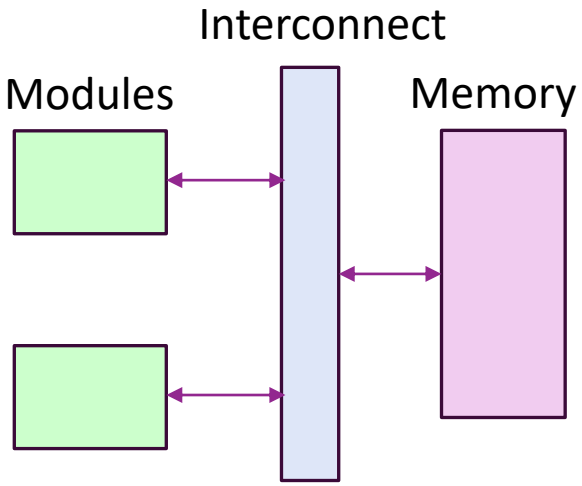
Method 1. Multi-Port memory

Each module has a dedicated port



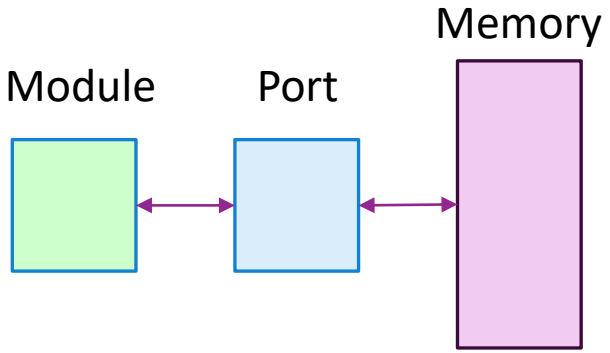
Method 2. Interconnect

Modules use connect infrastructure

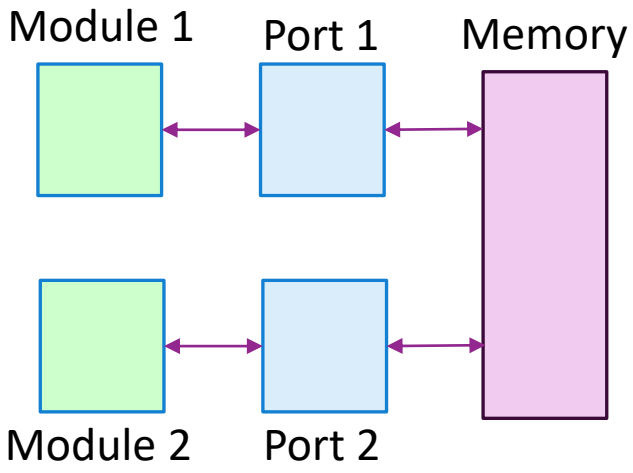


Single and Multi-Port Memories

- Single port
  - HW module has dedicated use of a memory
  - Memory has a single port
  - Port exposes address, data lines with address decoder
  - Does not permit transfer of data between modules



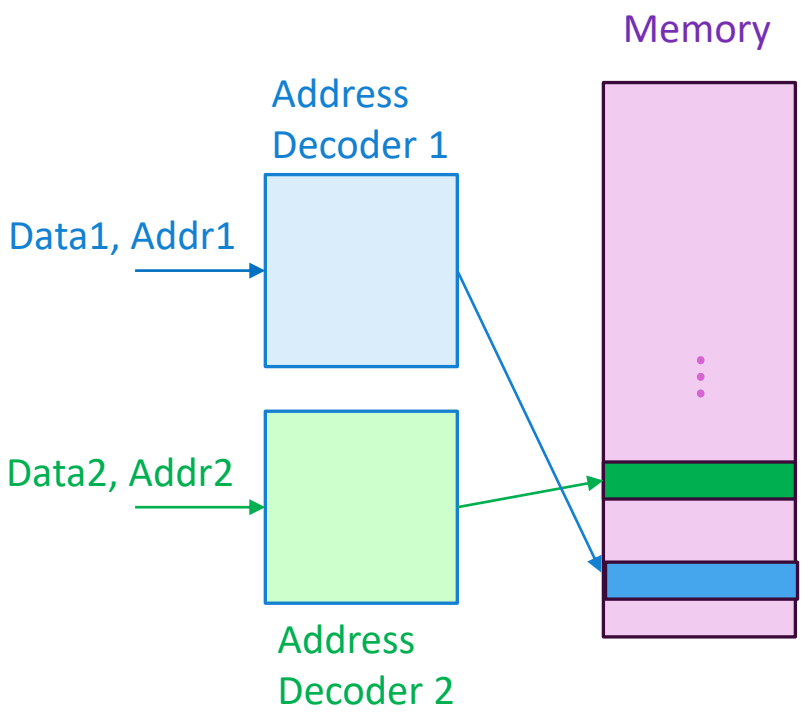
- Dual or multi-port
  - Multiple modules connect to the memory
  - Each module has a dedicated port
  - Allow simultaneous writes in same clock cycle
  - Most efficient



Problems with Multi-Port RAMs

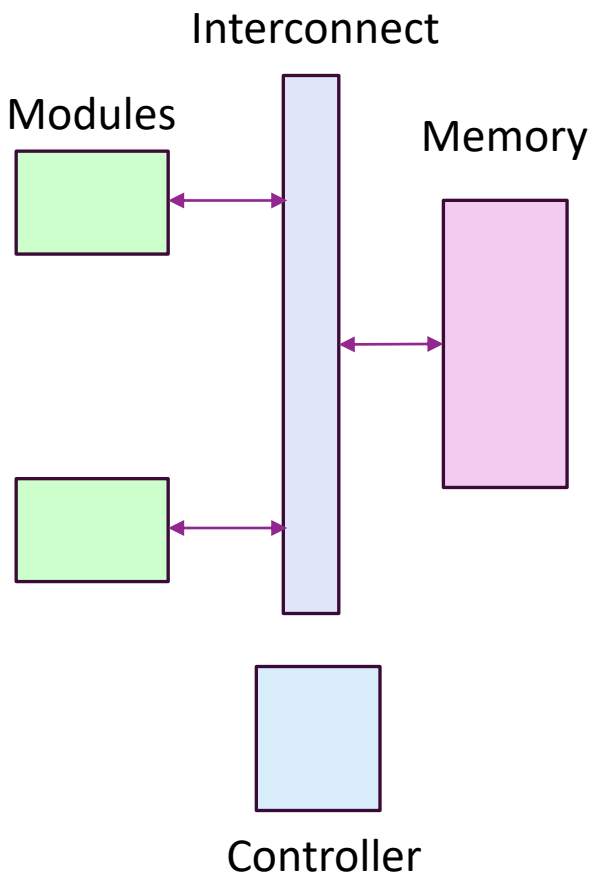
- Mutli-port SRAMs in ASICs are expensive
- Typically limited to small memorys
- Limited ports (e.g., 2R/2W)

Logical ports	Typical cell style	Rough transistors/bit	Rough area vs 6T
1R/1W (single-port)	6T SRAM cell	~6T	**1.0×**
1R/1W (2-port: 1R1W)	8T cell (separate read path)	~8T	**≈1.3×**
2R/1W	10T–12T cell	~10–12T	**≈1.7–2.0×**
2R/2W	12T–16T+ cell	~12–16T+	**≈2–2.7×+**
N-port (general)	custom multi-port SRAM	≈6T + O(N·ports)	**grows ~linear in ports**



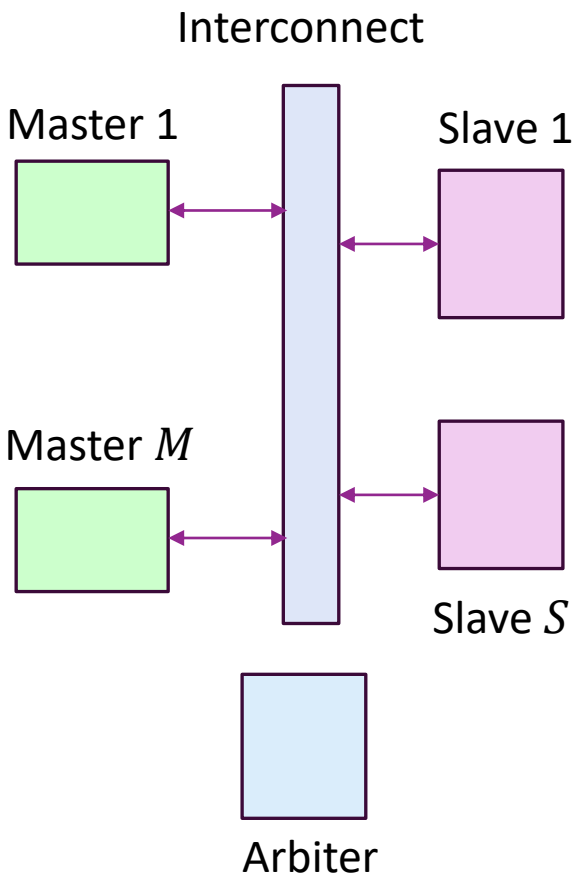
Interconnect

- Interconnect:
  - Physical and logical infrastructure for data communication
  - Between two or more functional units
  - Cores, memory controllers, I/O.
- Topology (Physical):
  - Wire layout or links (bus, mesh, crossbar, ring)
- Protocol (Logical):
  - Signals and timing for access (e.g., AXI4, TileLink, …)
- Arbitration (Control):
  - Method to resolve contention



Transactions, Master and Slaves

- Transactions:
  - A transfer of data over the interconnect
- Masters
  - Request transactions (reads or writes)
  - Ex: processors
  - Kernels that need to access memory
- Slaves
  - Respond to transactions (reads or writes)
  - Read transactions: Provide data to a master
  - Write transactions: Accept data from a master
- Arbiter
  - Determines which transaction is used in each cycle



Bus Topology

- One data path
- Only one master-pair link in each cycle
- Ex: ARM AHB (Advanced High-Performance Bus)
  - Pre-cursor to AXI
  - Traditional shared medium
  - Each master has dedicated side channel to arbiter
  - Arbiter selects a master and slave based on requests

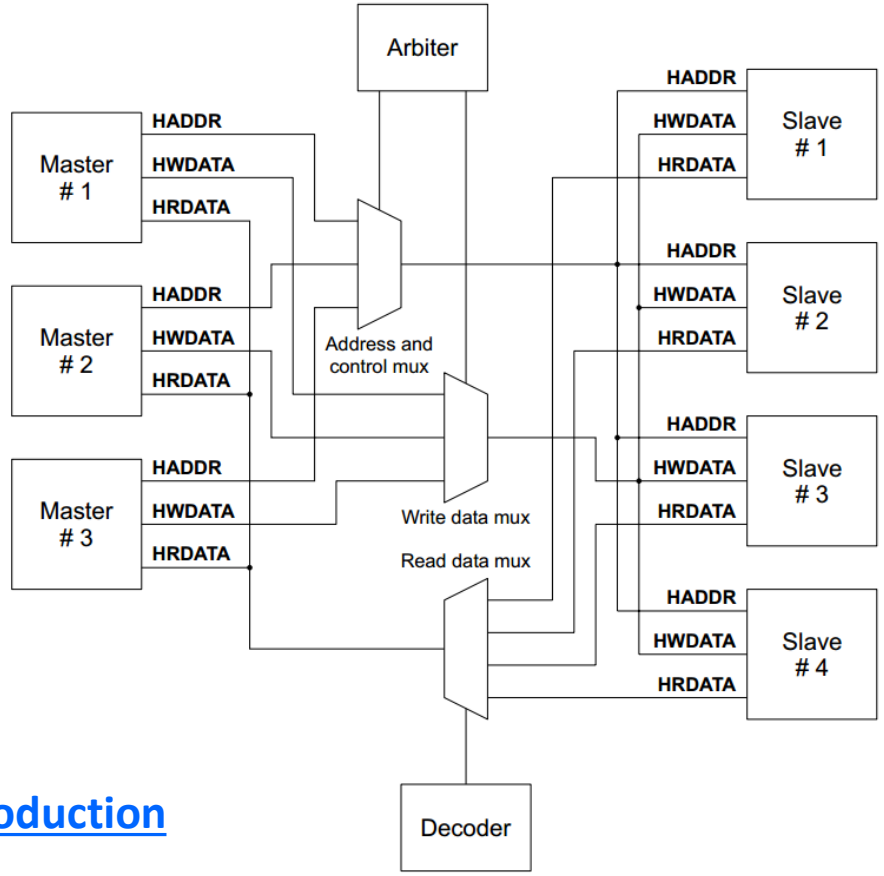
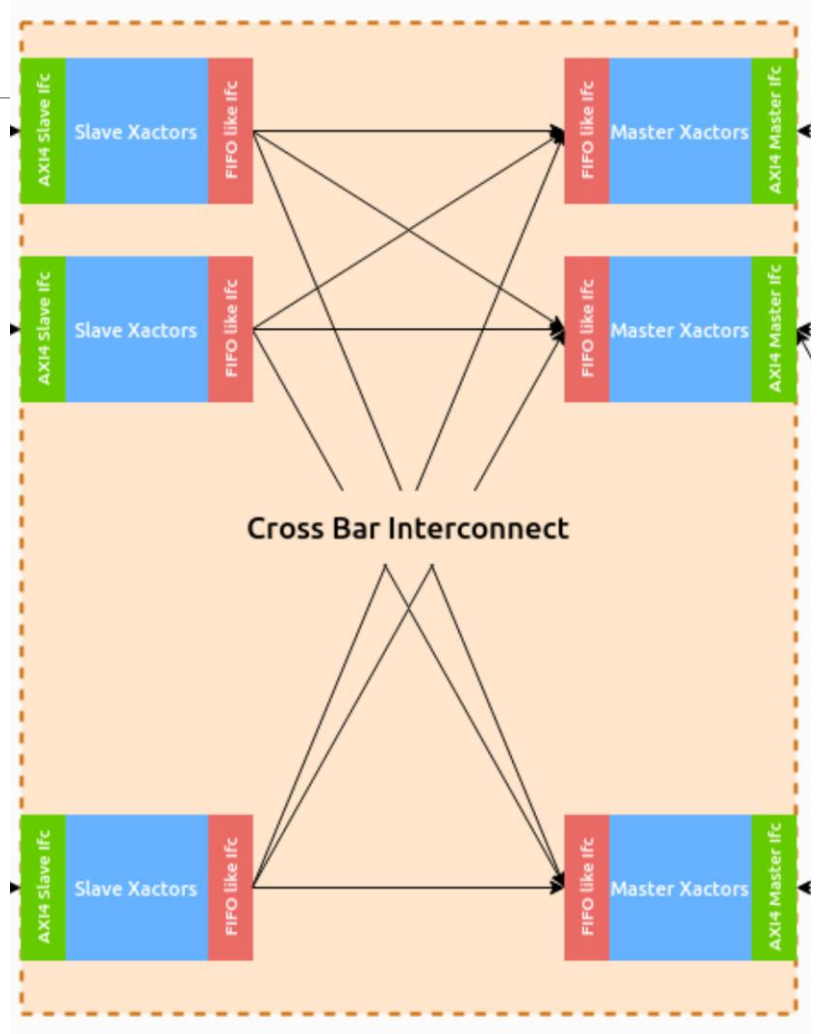


Figure from [The Advanced Microcontroller Bus Architecture: An Introduction]

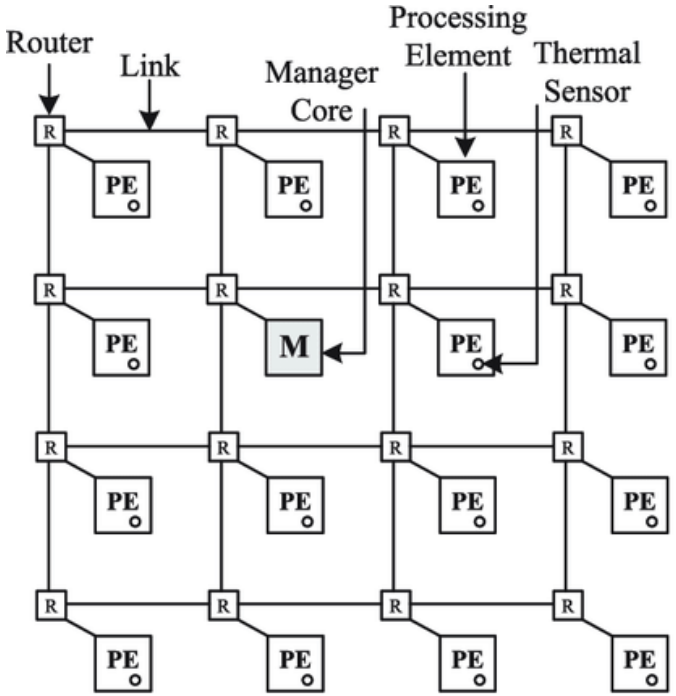
Crossbar Topology

- Connects:
  - M masters and S slaves
- Supports parallel master-slave transfers
  - Each master and slave can be involved in up to one transfer
  - Up to $\min(M, S)$ transfers simultaneously
  - Enables massive aggregate throughput
- Masters have separate request lines
- Common example: AXI4-MM protocol
  - See section later



Other Topologies

- Ring, Mesh, NoC
- Widely-used in large Systems on Chips
- Advanced topic beyond this class



(a) A 4 x 4 NoC platform

Figure from Chatterjee, Navonil, Suraj Paul, and Santanu Chattopadhyay. "Fault-tolerant dynamic task mapping and scheduling for network-on-chip-based multicore platform." *ACM Transactions on Embedded Computing Systems (TECS)* 16.4 (2017): 1-24.

Outline

- ❑ Shared Memory and Interconnects
- ❑ Implementing Simple Transactions over Shared Memory
- ❑ AXI4-Memory Mapped Protocol
- ❑ Implementing Shared Memory in Vitis HLS

Stream vs. Shared Memory Transactions

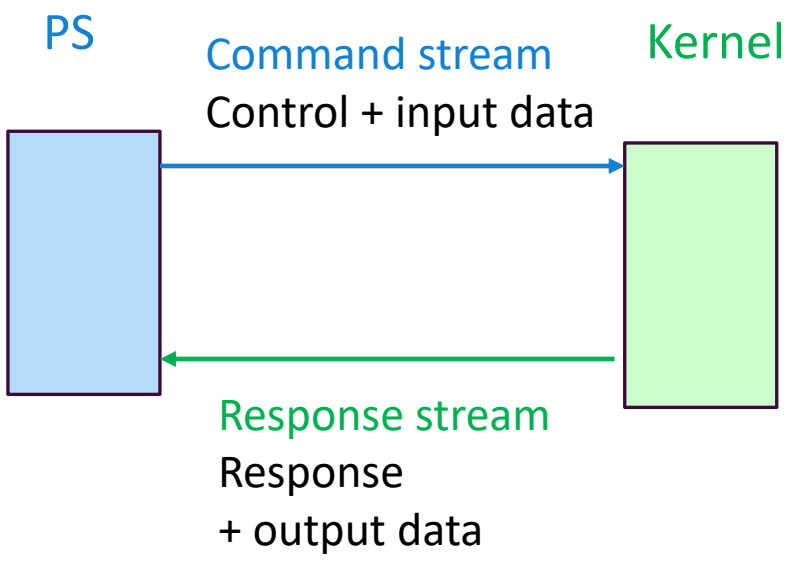
Dedicated stream transactions (Unit 5)

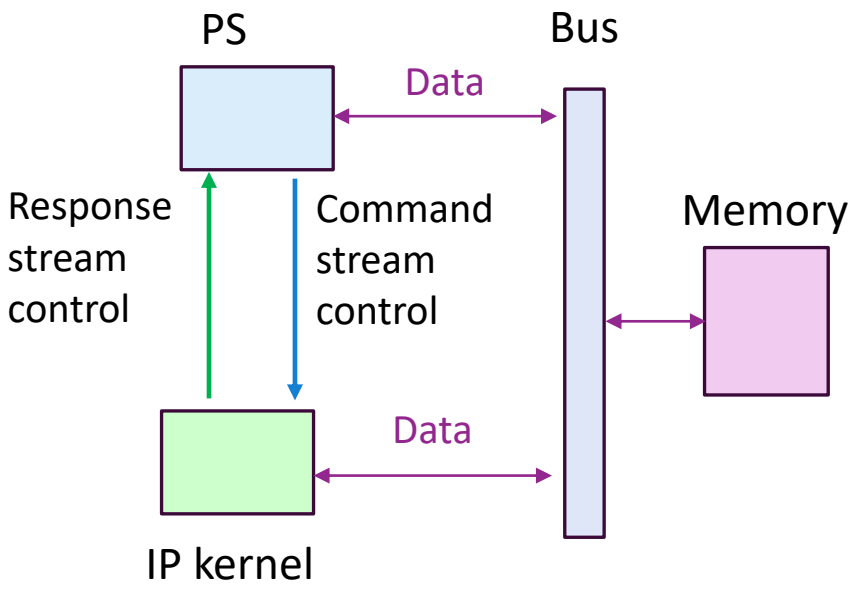
Send data & control on stream

Shared memory transaction (This unit)

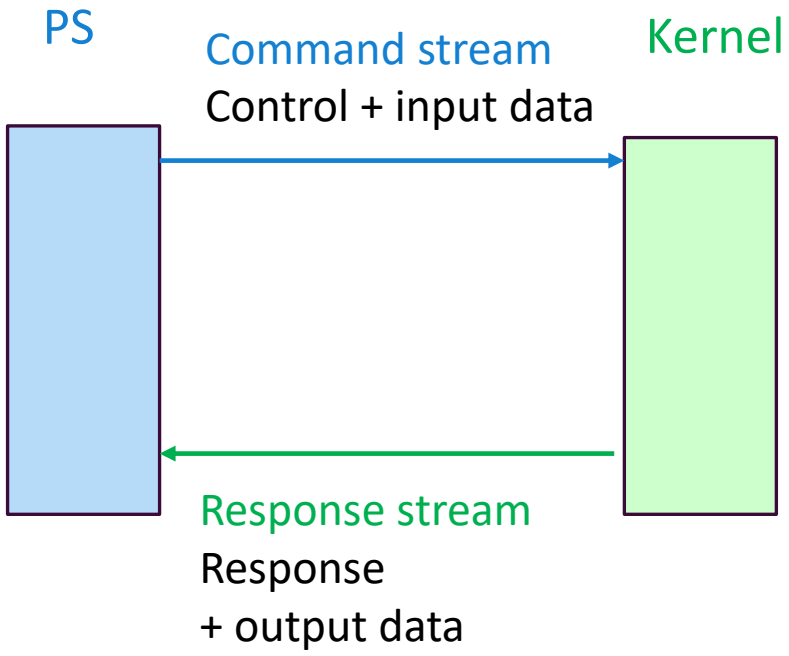
Send control on dedicated stream

Transfer data via shared memory



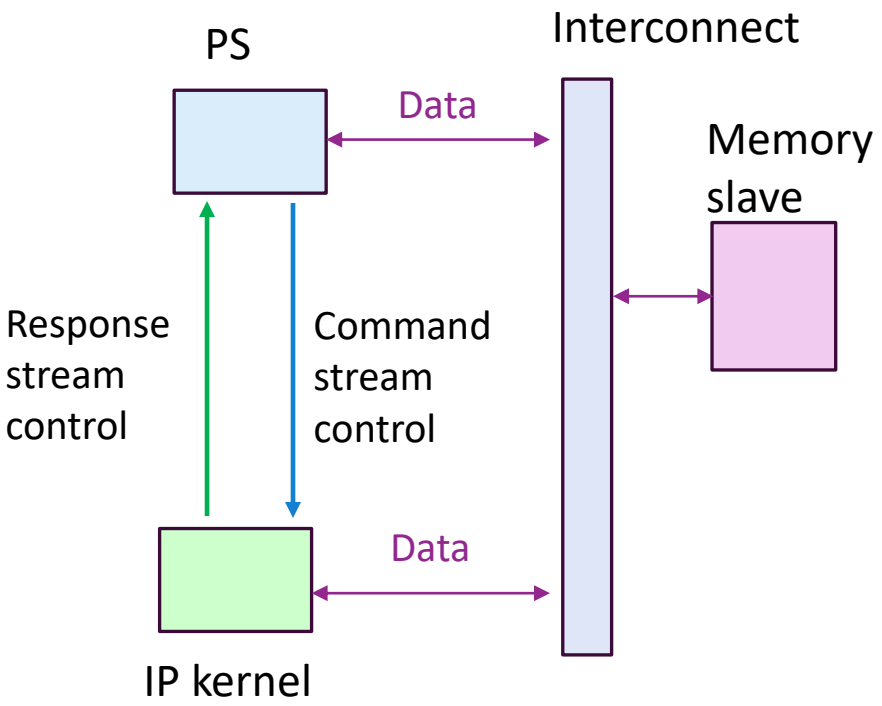


Stream Transfer (Unit 5)



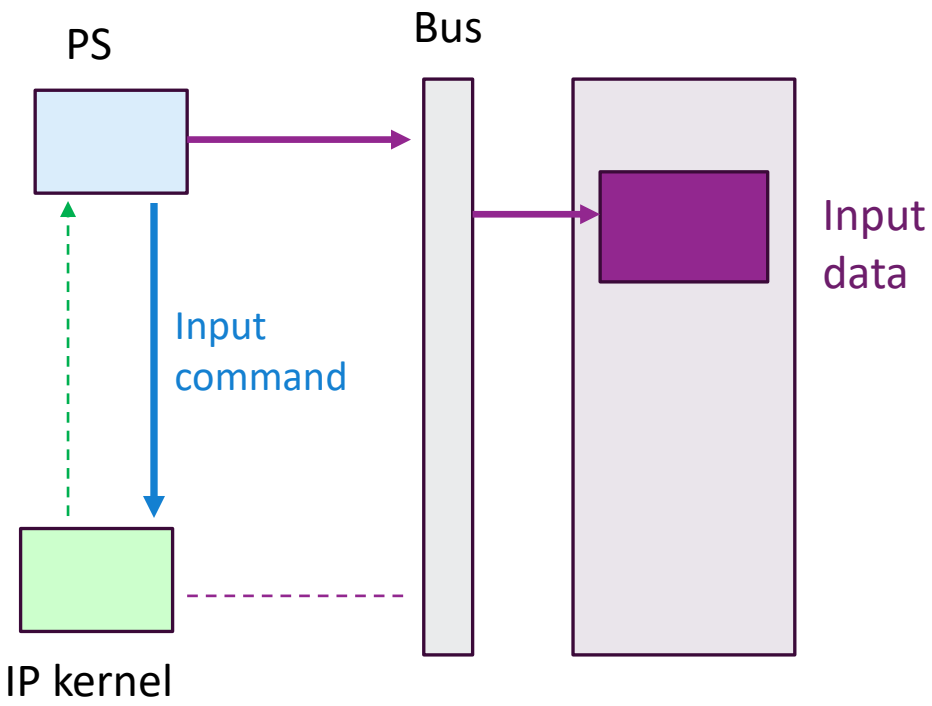
- Consider processing system (PS) sending job to kernel
- Use two streams
- Command stream: PS is master and sends to kernel
  - Command message descriptor
  - Input data to process
- Response stream: Kernel is master and sends to PS
  - Processed output data
  - Response descriptor (e.g., error codes, …)

Shared Memory Transaction



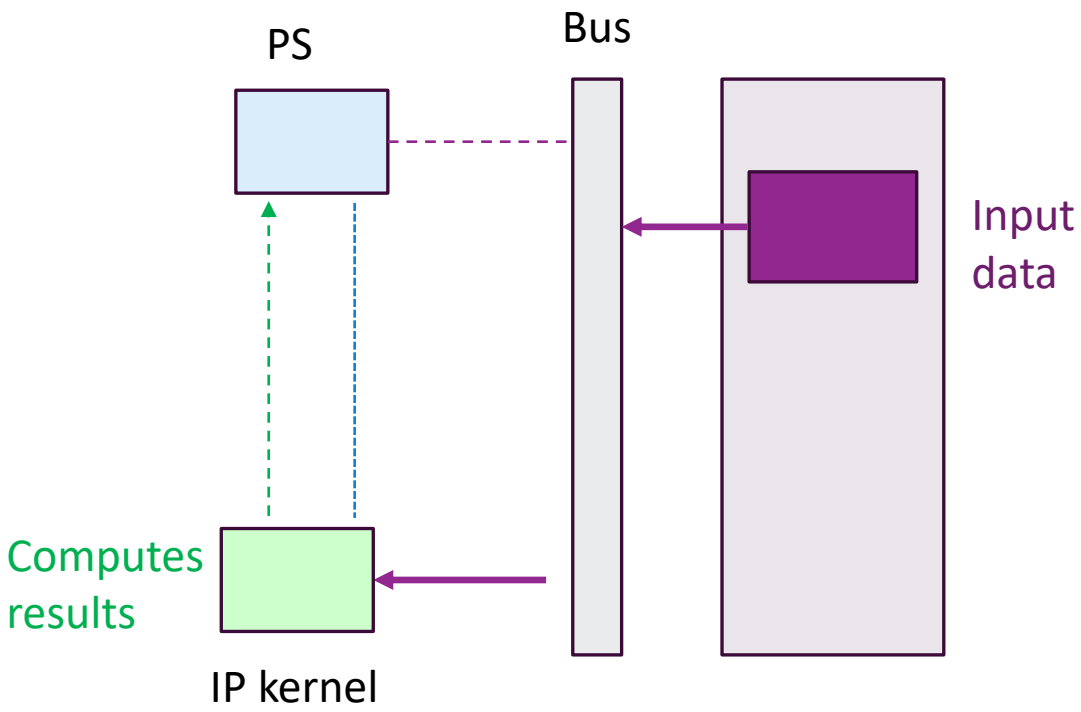
- PS and IP kernel both masters on a shared bus
- Memory is a slave
- Dedicated streams
  - Command stream: PS → kernel
  - Response stream: Kernel → PS
- Streams are used for control
- Right now, streams implemented on separate channels
  - Ex: AXI4-Stream
  - Later, we will show to map these into memory
- Memory and bus are used for data

Shared Mem Transaction: Command Phase



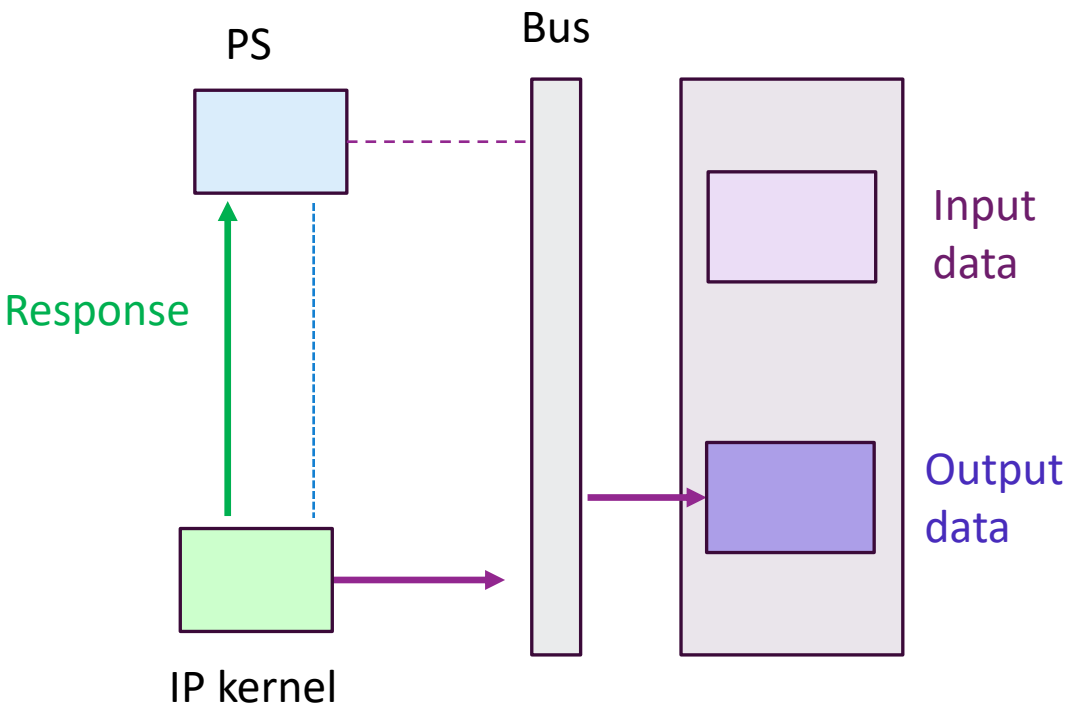
- PS writes input data into memory
  - Data to be processed
- PS sends input command to kernel
  - Input address: Where data resides
  - Output address: Where output should reside
  - Command info: What to do with data

Shared Mem Transaction: Compute Phase



- Kernel reads input data from memory
  - Uses input address sent in command
- Kernel processes input data

Shared Mem Transaction: Response Phase



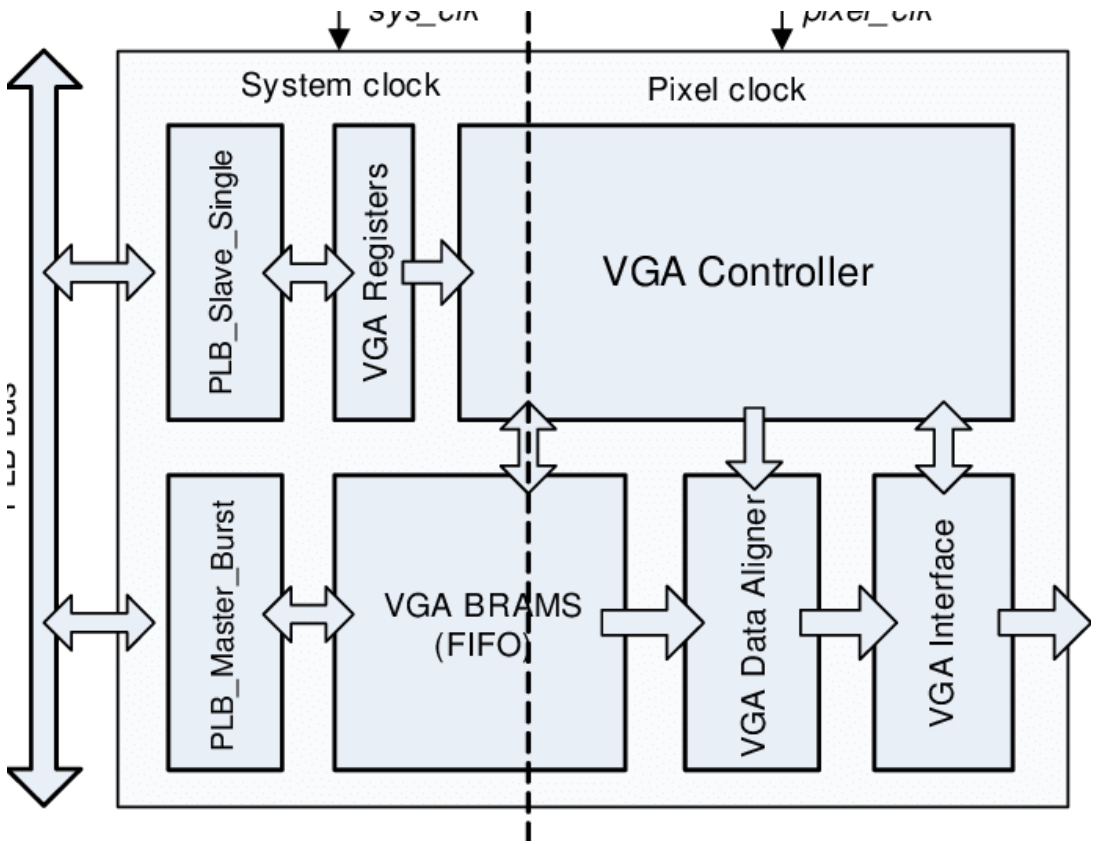
- Kernel writes output data to memory
  - Uses output address sent in command
- Kernel sends

Streaming vs. Shared Memory

	Streaming	Shared Memory
Access model	Sequential	Random
Control	Producer pushes data when available	Consumer pulls data when needed
Latency	Deterministic; minimal variation	Variable; depends on arbitration and usage
Arbitration	None if point-to-point	Required
Complexity		Simple design but adds
Resources	Dedicated resources per link, no statistical multiplexing	Shared resources, enables

Streaming Example: Simple Video Processing

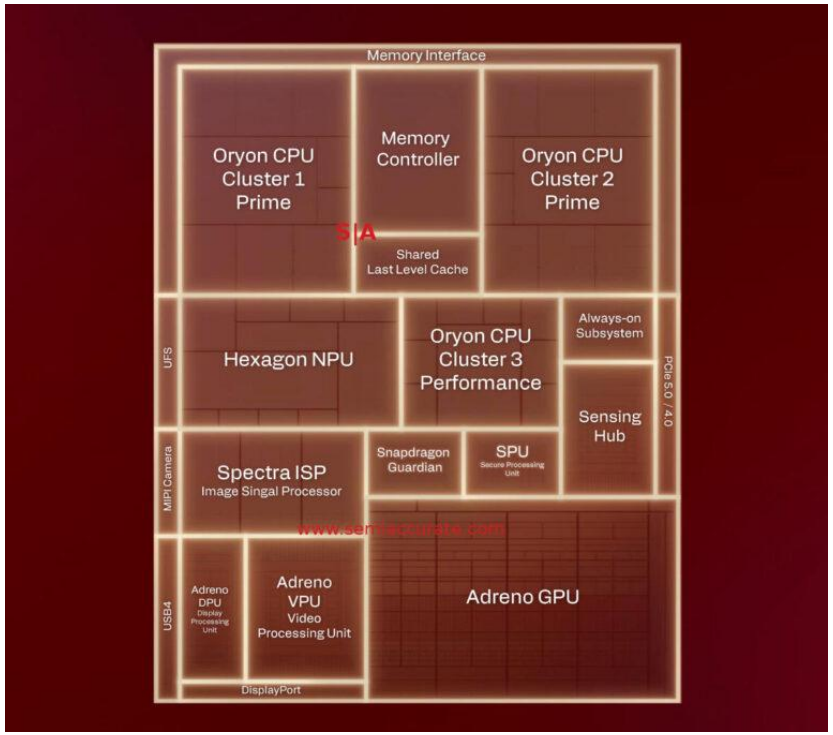
- Deterministic sequential flow
- Need dedicated high-speed links
- Buffering is memory intensive
- Need minimal reconfiguration



Tran, Van-Huan, and Xuan-Tu Tran. "An efficient architecture design for VGA monitor controller." *2011 International Conference on Consumer Electronics, Communications and Networks (CECNet)*. IEEE, 2011.

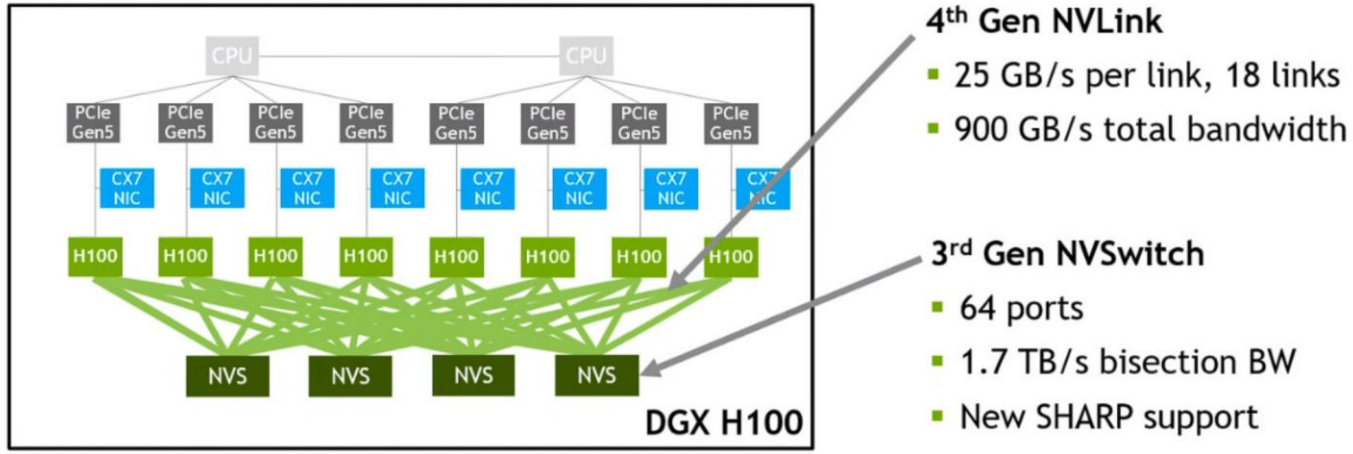
Shared Memory Ex: Mobile SoC

- Example:
  - Qualcomm Snapdragon, Apple MacBook M3
- NPU, CPU, GPU have access to common DDR memory
- CPU can write address to NPU, GPU
- NPU, GPU stores / retrieves data from memory
- Widely-used in virtually all SoC



Shared Memory Ex: Nvidia DGX H100

- LLMs stored in massive shared memory
- H100 GPUs fetch data as need
- Use high throughput NvLink



	A100 SuperPod			H100 SuperPod			Speedup	
	Dense PFLOP/s	Bisection [GB/s]	Reduce [GB/s]	Dense PFLOP/s	Bisection [GB/s]	Reduce [GB/s]	Bisection	Reduce
1 DGX / 8 GPUs	2.5	2,400	150	16	3,600	450	**1.5x**	**3x**

Outline

- ❑ Shared Memory and Interconnects
- ❑ Implementing Simple Transactions over Shared Memory
- ❑ AXI4-Memory Mapped Protocol
- ❑ Implementing Shared Memory in Vitis HLS

AXI Family of Protocols

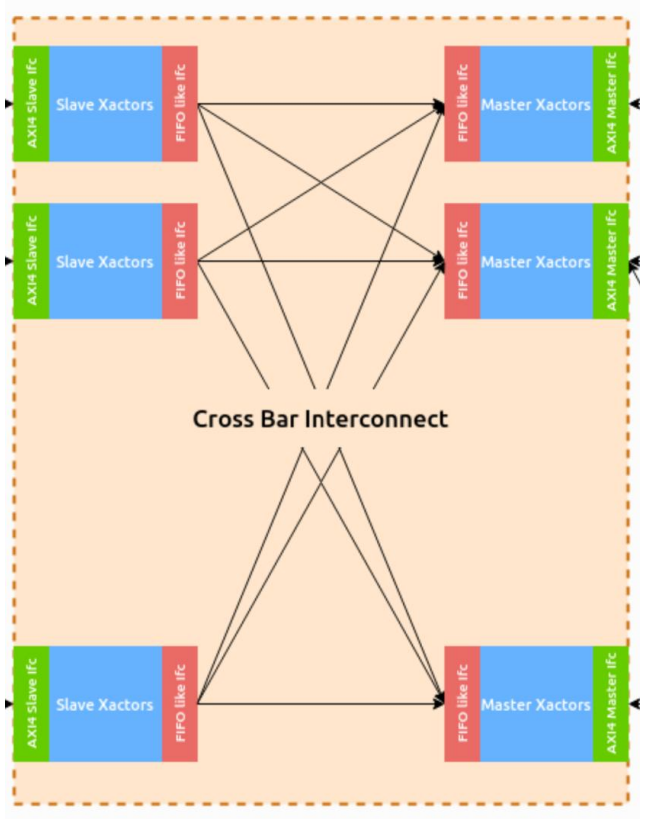
- AXI4-Stream (Covered in Unit 5)
  - Point-to-point link, flow control with TVALID / TREADY
  - 1 read/ write per clock cycle
  - Optional framing with TLAST
- AXI4-Lite (Covered in Unit 4)
  - Simple memory mapped protocol. Ideal for control registers
  - 1 read / 1 write per 3-5 clock cycles
- AXI4-Full (This unit).
  - Officially called AXI4 in AMD, but we use AXI4 for the family of protocols in this class
  - Builds on AXI4-Lite memory mapped
  - Handshaking signals to enable high throughput 1 read / write per clock cycle
- Terminology: We will use AXI-MM to mean both AXI4-Lite and AXI4-Full

AXI4 Memory Mapped Features

- Most widely-used interconnect fabric in industry
- Multiple outstanding requests
  - Master can issue several requests while others are ongoing
- Full-duplex with independent read and write channels
- Handshaking:
  - Independent VALID/READY for requests and data and read and write.
  - Provides flow control and modular design.
- Out of order protocols:
  - Flows marked with an ID. Order is preserved within an ID.
  - But controller may rearrange packet order for different IDs enabling prioritization
- Support for massive memories: DDR, HBM, not just on-chip RAM

AXI4-MM Architecture

- Interconnect is a crossbar
- Configurable number of masters and slaves
- Each master places transaction request
- Crossbar typically queues requests for each slave



Cost of Crossbars

- Logic cost:
  - Grows with product MS
  - But manageable relative to a large SoC
- Challenge in wiring
  - Wiring becomes dominant component of latency
  - Metal layer exhaustion
- Interconnects often 30-40% area in large SoCs

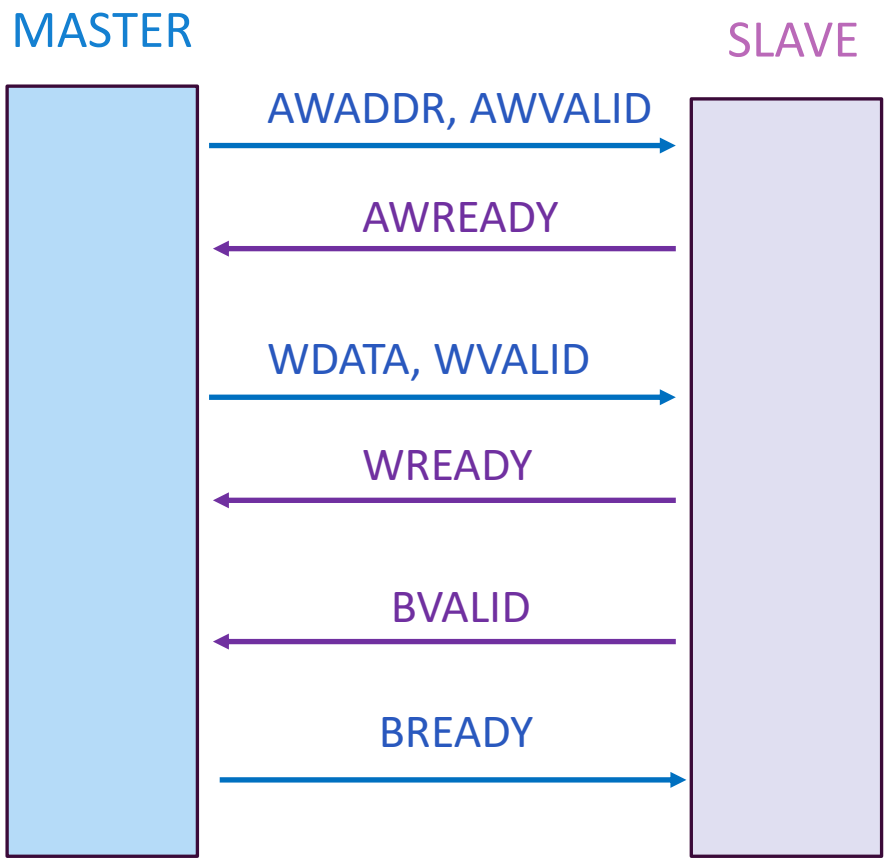
How data flows is key to high throughput design

Configuration	Est. Area (16nm)	Est. Gate Count (NAND2 equiv)
2 x 2	~0.008 mm²	~25k - 35k
4 x 4	**~0.035 mm²**	~100k - 140k
8 x 8	~0.140 mm²	~450k - 550k
16 x 16	~0.600 mm²	~2M+ (Serious congestion)

Logic estimates for AXI crossbar
64-bit address and data in TSMC 16 nm

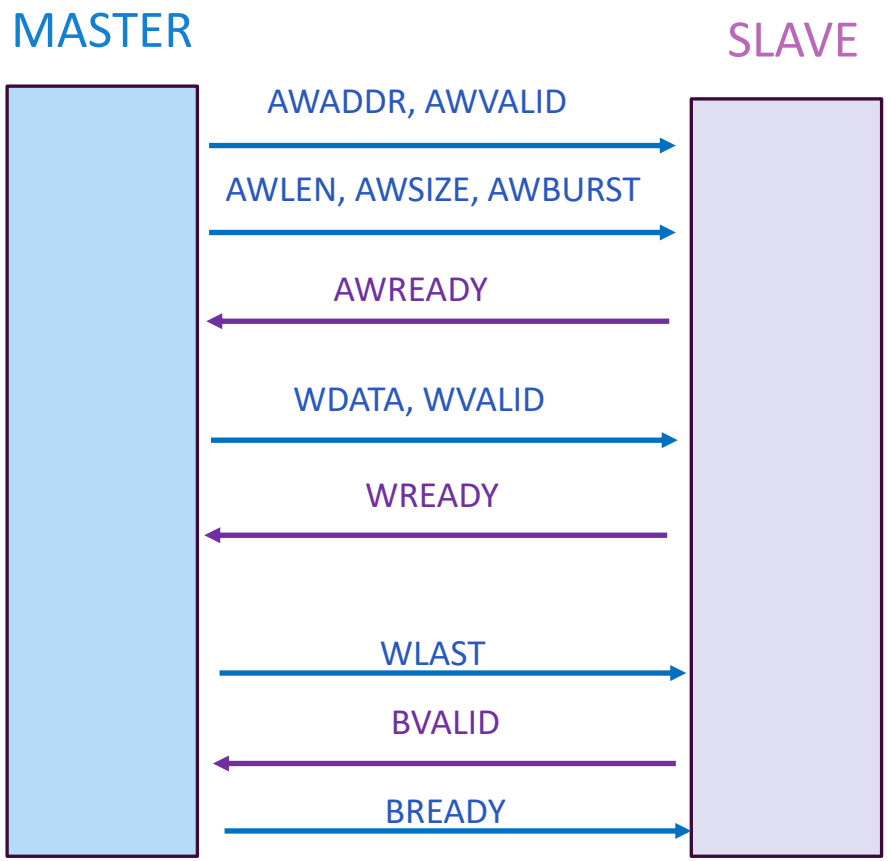
Review: AXI4-Lite Write Protocol

- Suppose master wants to write data D to address A
  - AWADDR=A and AWVALID=1
  - WDATA=D and WVALID=1
  - Both can occur in independent clock cycles
- When slave is ready for address or data it sets:
  - AWREADY=1 and WREADY=1
  - Slave can set these independently and any time it is ready
- Transfer:
  - When AWREADY=AWVALID=1 together address A is transferred
  - When WREADY=WVALID=1 together data D is transferred
- Handshake:
  - After both are transferred, slave set BREADY=1
  - Master sets BVALID=1 to acknowledge
  - Can start next transaction in same clock cycle as BVALID=1



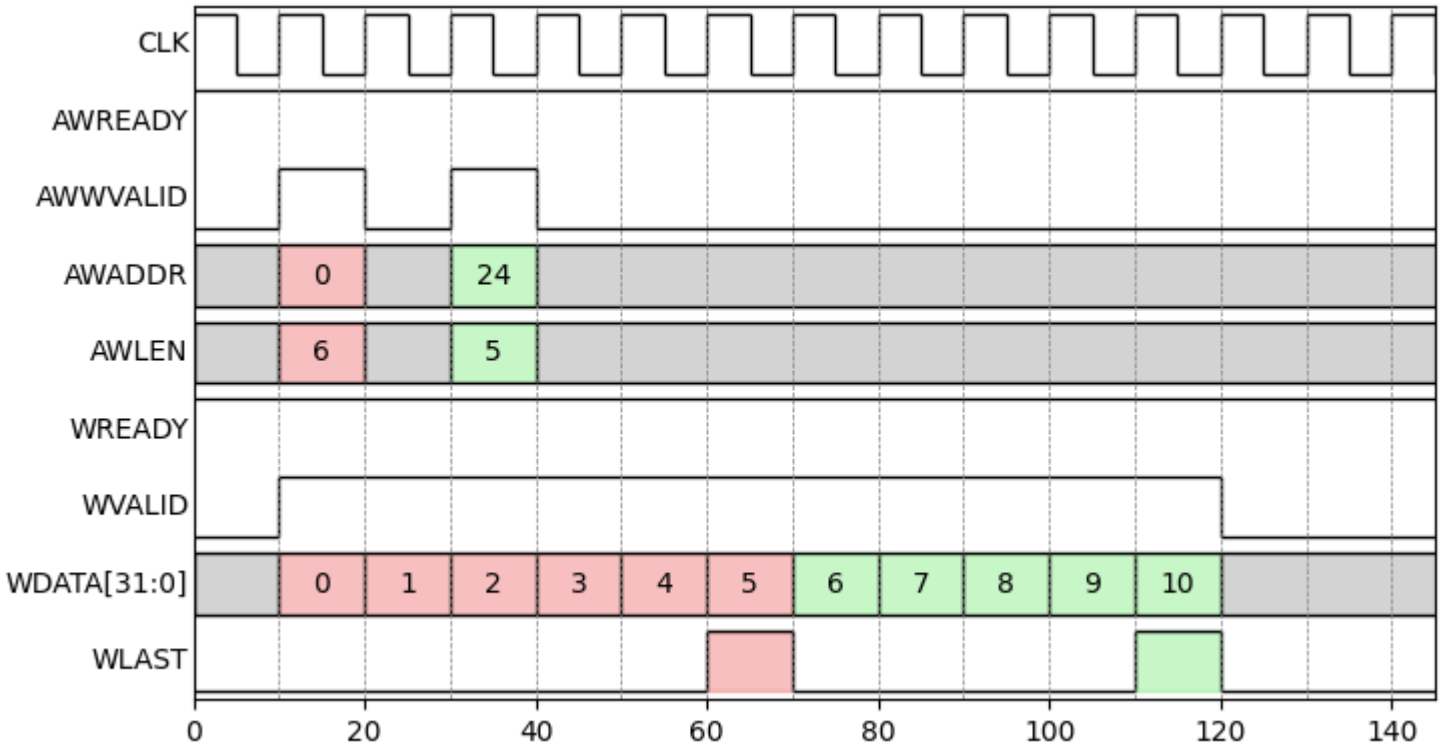
AXI4-Full Burst Signals

- AXI4-Full supports bursts of data
  - 1 cycle per read / write
  - Much faster than AXI4-Lite (typically 3-5 cycles)
- Additional signals sent on request:
  - AxLEN: Number of "beats" to transfer
  - AxSIZE: Width of transfer (for downward compatibility)
  - AxBURST: Type of burst (fixed or increment address)
- Transfer stage: Unchanged
  - Same WDATA, WVALID, WREADY
- Termination: Add WLAST
  - Indicates final beat (can terminate for AWLEN)



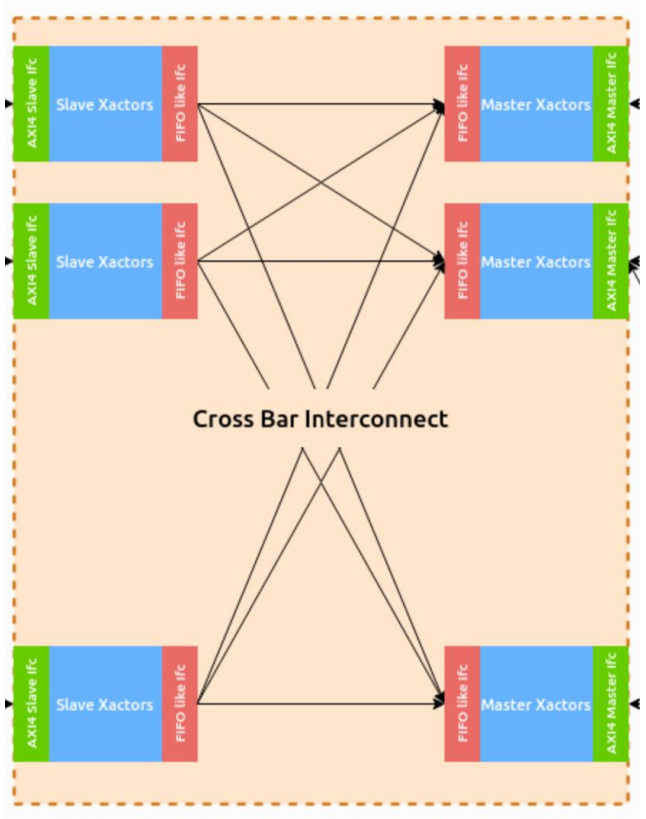
AXI4 Write Example

- Two write burst requests
  - Burst 0: AWLEN=0
  - Burst 1: AWLEN=5
- Request 1 early
  - Before write burst 0 completed
- WLAST at end of each burst



AXI4 Indexing

- Each transaction associated with an index
- Index is not unique
- Typical design:
  - Each master has a unique ID; or
  - One ID for priority class
- Crossbar will maintain order within an ID
- But may re-order between IDs
- Can be used for prioritization
- Advanced topic



Outline

- ❑ Shared Memory and Interconnects
- ❑ Implementing Simple Transactions over Shared Memory
- ❑ AXI4-Memory Mapped Protocol
- ❑ Implementing Shared Memory in Vitis HLS

Two Memory Interfaces in Vitis

- Native:
  - Typically used for direct connection to internal memories
  - Synthesized with a simple address decoder
  - Word indexed
  - Supported in Vivado with BRAM
- AXI4-MM
  - Typically used for DRAM or BRAM on a bus
  - Kernel connects to AXI4 memory controller (a block in Vivado)
  - AXI4-MM is always byte addressed

BRAM

- Creates native interface to BRAM
- AXI4-Lite
  - Read and write pointer
  - Length n
- Can read from memory in kernel

```
void kernel(
    ap_uint<64> mem[1024],  // The Native BRAM interface
    ap_uint<32> read_ind,   // Control registers (AXI4-Lite)
    ap_uint<32> write_ind,
    int n
) {
    // 1. Define the Native BRAM interface for the memory
    #pragma HLS INTERFACE bram port=mem

    // 2. Define the Control Interface (AXI4-Lite) for scalars and kernel control
    #pragma HLS INTERFACE s_axilite port=read_ind
    #pragma HLS INTERFACE s_axilite port=write_ind
    #pragma HLS INTERFACE s_axilite port=n
    #pragma HLS INTERFACE s_axilite port=return

    for (int i = 0; i < n; i++) {
        #pragma HLS PIPELINE II=1

        // Native access: Address is sent, data returns next cycle
        ap_uint<64> data = mem[read_ind + i];

        // ... do some computation ...

        mem[write_ind + i] = data;
    }
}
```

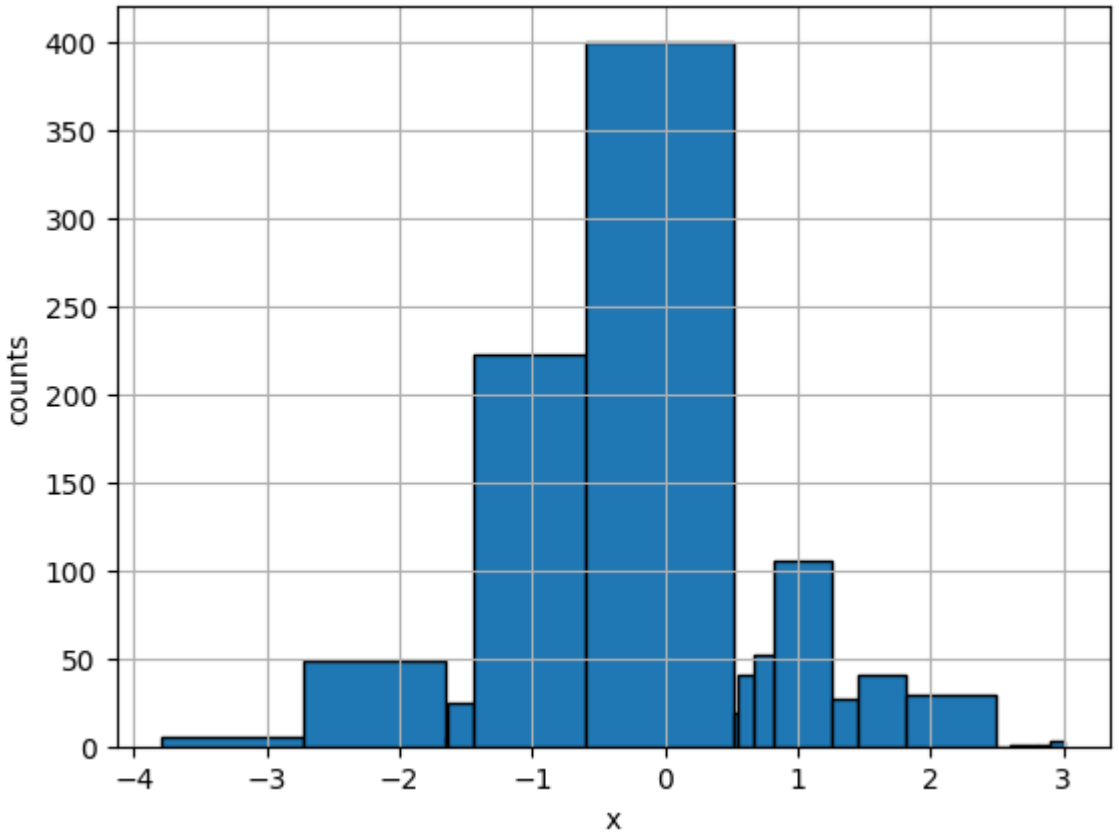
BRAM TB

- Creates BRAM emulation
  - Or actual memory

```
void kernel_tb() {

    ap_uint<64> mem[1024];  // The Native BRAM interface
    ap_uint<32> read_ind = 0;   // Control registers (AXI4-Lite)
    ap_uint<32> write_ind = 100;
    int n = 100;

    // Intialize memory with some data
    for (int i = 0; i < n; i++) {
        mem[read_ind + i] = ...;
    }

    kernel(mem, read_ind, write_ind, n);

    // Check results in mem[write_ind ... write_ind+n-1]
    for (int i = 0; i < n; i++) {
        assert(mem[write_ind + i] == ...);
    }
)
```

AXI4 Histogram Example

- Non-uniform histogram
- Given:
  - N data values $data[i]$
  - B bin edges $bin_edges[j]$
- Compute
  - $counts[j],\ j = 0, \ldots, B$
  - Includes underflow and overflow values



Histogram Command Input

- Gives input addresses:
  - Data
  - Bin edges
- Output addresses
  - Counts
- Transaction IDs and lengths

```python
class HistCmd(DataList):
    """Command descriptor for the histogram accelerator."""

    elements = {
        "tx_id": {
            "schema": TxIdField,
            "description": "Transaction ID for correlating command and response",
        },
        "data_addr": {
            "schema": AddrField,
            "description": "Base address of the input data buffer",
        },
        "bin_edges_addr": {
            "schema": AddrField,
            "description":("Base address of the output histogram bin edges.  "
                "There should be nbins-1 edge values.  "
                "bin 0 will have values x < bin_edges[0], bin i will have values "
                "bin_edges[i-1] <= x < bin_edges[i], and the last bin will have values "
                "x >= bin_edges[nbins-2]"),
        },
        "ndata": {
            "schema": NdataField,
            "description": "Number of input data elements to histogram",
        },
        "nbins": {
            "schema": NbinField,
            "description": "Number of histogram bins to produce",
        },
        "cnt_addr": {
            "schema": AddrField,
            "description": "Base address of the output histogram counts buffer",
        },
    }
    include_dir = INCLUDE_DIR
```

Histogram Interface

- AXI4 streams
  - For command and response
  - These could have been memory-mapped
- AXI4-Full access to memory
- Read command from AXI4 stream

```
void hist(
    hls::stream<axis_word_t>& in_stream,
    hls::stream<axis_word_t>& out_stream,
    mem_word_t* mem) {
#pragma HLS INTERFACE axis port=in_stream
#pragma HLS INTERFACE axis port=out_stream
#pragma HLS INTERFACE m_axi port=mem offset=slave bundle=gmem depth=max_mem_words
#pragma HLS INTERFACE ap_ctrl_hs port=return

    HistCmd cmd;
    streamutils::tlast_status cmd_tlast = streamutils::tlast_status::no_tlast;
    cmd.read_axi4_stream<stream_dwidth>(in_stream, cmd_tlast);

    HistResp resp;
    resp.tx_id = cmd.tx_id;
    resp.status = HistError::NO_ERROR;

    static float data_buf[max_ndata];
    static float edge_buf[(max_nbins > 1) ? (max_nbins - 1) : 1];
    static ap_uint<32> count_buf[max_nbins];

    const int ndata = static_cast<int>(cmd.ndata);
    const int nbins = static_cast<int>(cmd.nbins);
```

Histogram Read Phase

- Read data and bin edges to a local buffer
- Local buffer is not strictly needed for data
  - Easy to illustrate

```
    const int data_word_idx = memmgr::byte_addr_to_word_index<mem_dwidth>(cmd.data_addr);
    const int edge_word_idx = memmgr::byte_addr_to_word_index<mem_dwidth>(cmd.bin_edges_addr);
    const int count_word_idx = memmgr::byte_addr_to_word_index<mem_dwidth>(cmd.cnt_addr);

    float32_array_utils::read_array<mem_dwidth>(mem + data_word_idx, data_buf, ndata);
    if (nbins > 1) {
        float32_array_utils::read_array<mem_dwidth>(mem + edge_word_idx, edge_buf, nbins - 1);
    }

    for (int i = 0; i < max_nbins; ++i) {
#pragma HLS PIPELINE II=1
        count_buf[i] = 0;
    }
```

Core Histogram Loop

- Nested loop
- Outer loop is not pipelined
- Must be careful of inner loop pipelining
- Will show synthesis reports in class

```
    // For each sample, find the appropriate bin and increment the count.
    // Note that only the inner loop that searches for the correct bin is
    // pipelined with II=1.
    for (int i = 0; i < ndata; ++i) {
#pragma HLS LOOP_TRIPCOUNT min=1 max=max_ndata

        float sample = data_buf[i];
        int bin = 0;

        hist_search:  for (int b = 0; b < nbins - 1; ++b) {
#pragma HLS LOOP_TRIPCOUNT min=1 max=max_nbins
#pragma HLS PIPELINE II=1
            if (sample >= edge_buf[b])
                bin = b + 1;
        }

        count_buf[bin] = count_buf[bin] + 1;
    }
```

Testbench Memory Allocation

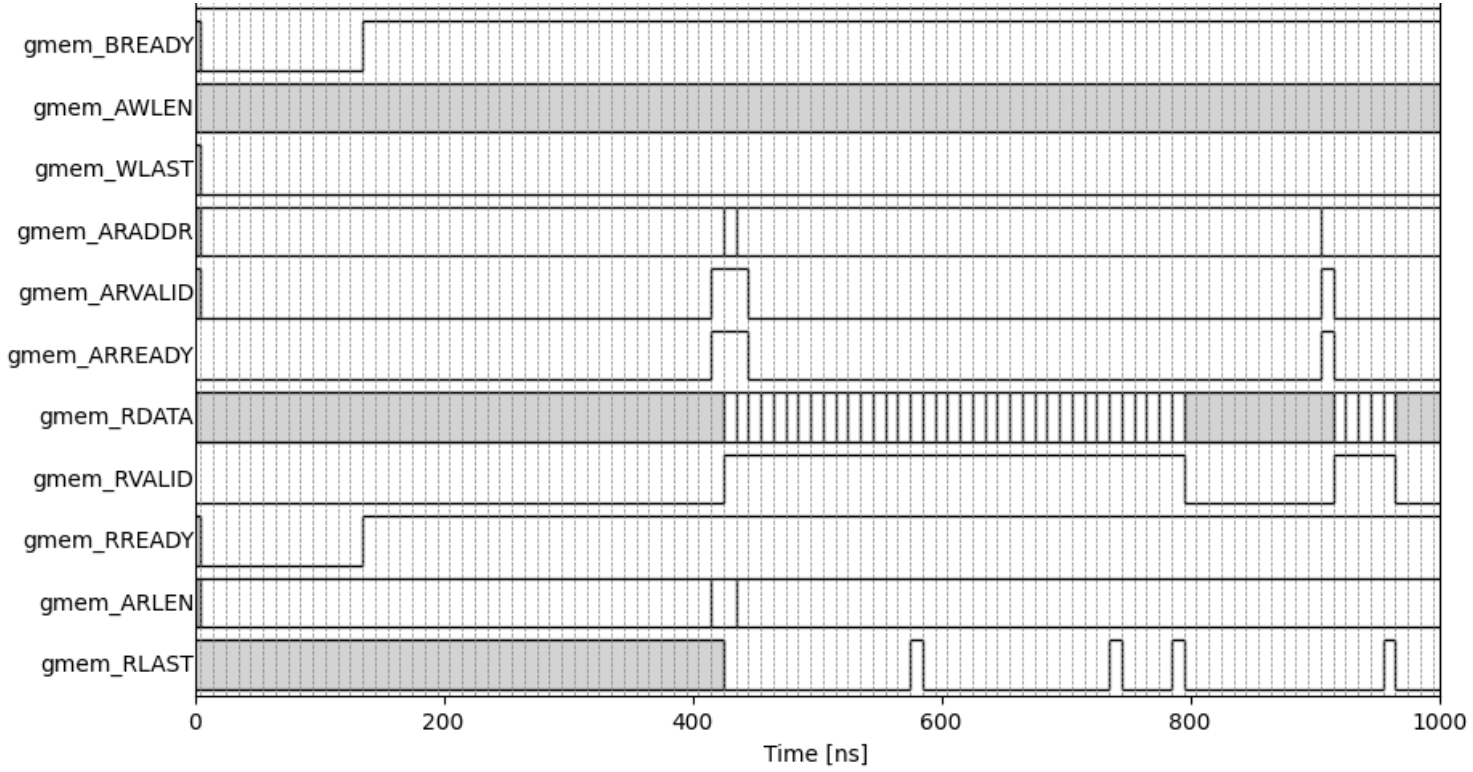
- Testbench creates memory
- Allocates blocks in memory
- Computes byte addresses

```
// Allocate regions in the flat memory array using MemMgr, mirroring the Python Memory.alloc flow.
static mem_word_t mem[MEM_SIZE] = {};
pysilicon::memmgr::MemMgr<mem_dwidth> mgr(mem, MEM_SIZE);

// Word counts for each region in the flat memory array.
const int nwords_data  = float32_array_utils::get_nwords<mem_dwidth>(ndata);
const int nwords_edges = (nbins > 1) ? float32_array_utils::get_nwords<mem_dwidth>(nbins - 1) : 1;
const int nwords_count = uint32_array_utils::get_nwords<mem_dwidth>(nbins);

const int data_word_idx  = mgr.alloc(nwords_data);
const int edge_word_idx  = mgr.alloc(nwords_edges);
const int count_word_idx = mgr.alloc(nwords_count);

// Byte addresses used in the HistCmd (AXI4 byte-addressed interface).
const int bytes_per_word   = mem_dwidth / 8;
const ap_uint<mem_awidth> data_byte_addr  = data_word_idx  * bytes_per_word;
const ap_uint<mem_awidth> edge_byte_addr  = edge_word_idx  * bytes_per_word;
const ap_uint<mem_awidth> count_byte_addr = count_word_idx * bytes_per_word;
```

Testbench Command

- Writes addresses in command
- Upon receipt of response
  - Retieves data
  - Compares against test vectors

```
// Build the HistCmd in the testbench using the allocated addresses.
HistCmd cmd;
cmd.tx_id         = tx_id;
cmd.data_addr     = data_byte_addr;
cmd.bin_edges_addr = edge_byte_addr;
cmd.ndata         = ndata;
cmd.nbins         = nbins;
cmd.cnt_addr      = count_byte_addr;

// Push the command descriptor into the input stream and run the kernel.
hls::stream<axis_word_t> in_stream;
hls::stream<axis_word_t> out_stream;
cmd.write_axi4_stream<stream_dwidth>(in_stream, true);
hist(in_stream, out_stream, mem);
```

Timing: Read Phases

- Test parameters
  - 7 bins
  - 37 data points
- Partitioning
  - Data broken into three busts
  - 16, 16, and 5
  - Requests given at beginning